

Workshop 3 – Kaggle

Mechanisms of Action (MoA) Prediction – Kaggle Competition

Course: Systems Analysis

Laura Natalia Pardo Castillo – 20221020146

Carmen Sofía Florez Juajibioy – 20221020140

Ángela Sofía Puentes Medina – 20241020114

Carlos Julian Rojas Romero – 20241020039

November 8, 2025

Contents

1	Introduction	2
2	Robust System Design	2
2.1	Applied design principles (modularity, scalability, fault tolerance, maintainability, usability)	2
2.2	Updated architecture diagram	3
2.3	Description of system components	4
3	Quality and Risk Analysis	4
3.1	Risk or failure point identification	4
3.2	Impact and likelihood assessment	5
3.3	Mitigation and monitoring strategies	5
4	Project Management Plan	6
4.1	Team roles and responsibilities	6
4.2	Main milestones and deliverables	6
4.3	Methodology or management tool used	7
4.4	Schedule and workflow	7
5	Incremental Improvements	8
6	Final Reflection	9
7	Conclusions	10
8	References	11

1 Introduction

In this workshop, the goal is to strengthen the design of the Mechanisms of Action (MoA) prediction system developed in earlier phases, applying principles of robust engineering and project management strategies. The focus is on refining the architecture; improving reliability, scalability, maintainability, and usability; and ensuring that design decisions comply with quality standards such as ISO 9000, CMMI, and Six Sigma.

The system already has a modular architecture organized into five main modules: data loading, preprocessing, training, prediction, and report generation. During this workshop, each component was reviewed to increase its fault tolerance, scalability, and clarity, incorporating practices such as version control on GitHub, preserving results, and standardizing data formats. These decisions reflect the competence gained in planning and executing robust software projects.

In addition, a quality and risk analysis was conducted, identifying potential points of failure such as errors in data preparation, model overfitting, and limitations in biological interpretability. For each risk, mitigation and monitoring strategies were defined, demonstrating the ability to apply best engineering practices and ensure the system's effectiveness during its development and operation. Finally, the evolution of the design and project management is documented, incorporating incremental improvements based on feedback from previous workshops and the experience gained.

2 Robust System Design

2.1 Applied design principles (modularity, scalability, fault tolerance, maintainability, usability)

The system is built by applying the principles of modularity, scalability, fault tolerance, maintainability, and usability in a practical way.

Modularity. The system is organized into five main modules:

- Data loading
- Preprocessing
- Model training
- Prediction
- Report generation

Each module is implemented as an independent script or notebook, allowing it to be run, tested, and updated separately. This modular structure makes it easy to replace the

model or dataset without modifying the rest of the system. For example, the training module can switch between a Scikit-learn Random Forest and a neural network adapted from a Kaggle notebook.

Scalability. Although the system runs locally, its modular structure allows the workflow to be expanded by adding new datasets or integrating more complex models without changing the overall architecture. This flexibility supports the goal of building an extensible pipeline that can evolve and incorporate additional stages (such as feature selection or cross-validation) as the project progresses or additional experiments are required.

Fault tolerance. To ensure data integrity and avoid overwrites, each run generates separate output files (e.g., `results.csv`, `submission.csv`, and model metrics). This mechanism preserves previous versions and allows performance comparisons across experiments, aligning with the simulation and validation principles covered in Workshop No. 4. Likewise, the use of simple, portable formats (CSV and models serialized with `joblib`) enables recovery of results in case of failures or interruptions during execution.

Maintainability. The source code is organized by modules (folders `/data`, `/models`, `/reports`) and managed in a GitHub repository. Each team member participates in a part of the workflow (data, modeling, evaluation, or reporting), applying the project management and version control best practices from Workshop No. 3. This organization makes it possible to track changes, detect errors, and apply iterative improvements, ensuring controlled and collaborative development.

Usability. The workflow is designed so that a user can easily reproduce the entire process, from data loading to generating predictions and reports. Usage instructions are documented in the `README.md` file, with clear steps to run the notebooks or scripts in order. In this way, the system meets the objective of offering a practical, accessible tool aligned with the dynamics of Kaggle competitions.

2.2 Updated architecture diagram

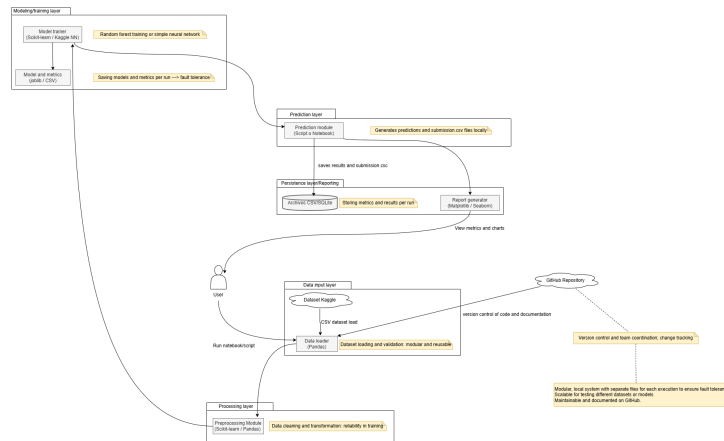


Figure 1: Updated system architecture diagram.

2.3 Description of system components

Componente	Función principal	Implementación práctica
Data Loader	Carga y validación del dataset descargado desde Kaggle.	Pandas (<code>read_csv</code>); verificación de columnas y manejo de valores faltantes.
Preprocessing Module	Limpieza y transformación de variables para el modelo.	Pandas y Scikit-learn (<code>get_dummies</code> , <code>StandardScaler</code>).
Model Trainer	Entrenamiento del modelo predictivo y registro de métricas.	Scikit-learn (<code>RandomForest</code> , <code>LogisticRegression</code>) o red neuronal sencilla adaptada desde Kaggle.
Prediction Module	Generación de predicciones y exportación del archivo de envío.	Script o notebook que carga el modelo (<code>joblib</code>), realiza las predicciones y guarda <code>submission.csv</code> .
Report Generator	Visualización de métricas y gráficos de comparación.	Matplotlib / Seaborn; exporta gráficos o tablas de resultados.
Repositorio GitHub	Control de versiones, documentación y coordinación del equipo.	Uso de ramas, <i>commits</i> descriptivos y registro de avances en <code>README.md</code> .

Table 1: Componentes del sistema, función principal e implementación práctica

3 Quality and Risk Analysis

In this section, we identify the key risks associated with the design and development of the Mechanisms of Action (MoA) prediction system, assess their impact, and propose mitigation and monitoring strategies throughout development and operation. The goal is to ensure the system’s reliability, maintainability, and usability, while taking into account the team’s constraints and the time available.

3.1 Risk or failure point identification

- **Data quality and preparation.** Genomic data exhibit high dimensionality and significant class imbalance across mechanisms of action. If errors or missing values are present, the model may learn incorrect or inconsistent patterns, leading to

unreliable predictions.

- **Model overfitting.** Given the large number of features and the limited number of samples, complex models may memorize the training data rather than generalize to new compounds, undermining the validity and usefulness of predictions.
- **Interpretability and biological validation.** Complex models can be difficult for domain experts to interpret, limiting the ability to experimentally validate predicted mechanisms and to prioritize improvements.
- **Pipeline integration and operation.** The end-to-end workflow—data loading, preprocessing, training, prediction, and report generation—can be disrupted by data transfer errors or format inconsistencies, causing delays or incorrect results.

3.2 Impact and likelihood assessment

Each of these risks has a significant impact on the system’s reliability and accuracy, as well as on ease of use and maintenance. The likelihood of occurrence is considered moderate, given the data volume, model complexity, and the limitations of a small team. Overfitting and pipeline errors can produce compounding effects, undermining both result quality and biological validation.

3.3 Mitigation and monitoring strategies

- **Data quality.** Perform exploratory analysis during data loading and preprocessing: detect missing values, identify outliers, and normalize features. Log basic quality metrics and raise early alerts for inconsistencies.
- **Model overfitting.** Apply cross-validation and hyperparameter tuning (e.g., depth and number of trees for Random Forest). For simple neural networks adapted from Kaggle, use early stopping to halt training when the validation metric stops improving. Track all model versions with their associated metrics to enable auditing and result comparison.
- **Interpretability and biological validation.** Use interpretation techniques such as SHAP or LIME to identify which features influence each prediction. This supports expert review of biological consistency and helps prioritize model improvements. Include visualizations of feature importance in reports.
- **Pipeline integration and operation.** Implement input/output controls in each module to detect errors before they propagate. The prediction service should verify that incoming data meet expected formats, while storage and reporting modules confirm correct persistence of results and metrics.

- **Continuous monitoring.** During development and operation, adopt a supervision approach based on periodic reviews and lightweight automated checks. Each module performs basic consistency validations; training runs are monitored with plots of validation-metric trends; and any detected issues are recorded in organized logs to enable quick identification and correction. This ensures the system meets the established quality standards.

4 Project Management Plan

To ensure a structured and efficient development of the system, roles, responsibilities, management tools, and a schedule with weekly milestones were defined to enable tracking and ensure the project’s objectives are met.

4.1 Team roles and responsibilities

Each team member has a primary role with clearly assigned responsibilities, while retaining the ability to collaborate on secondary tasks as project needs arise.

Role	Owner	Main function
Data Lead	Person 1	Data preparation and cleaning, quality audits, and exploratory data analysis.
Modeling Lead	Person 2	Model training, cross-validation, and performance monitoring.
Explainability & Reporting Lead	Person 3	Implementation of interpretability techniques (SHAP/LIME), dashboard creation, and visualization of results.
Documentation & Project Management Lead	Person 4	Progress tracking, task coordination, Kanban board management, and compilation of deliverables.

Table 2: Team roles, owners, and main functions

4.2 Main milestones and deliverables

Development is organized over a five-week period, with concrete deliverables and designated owners for each stage.

Week	Milestone	Deliverable	Owner
1	Data Preparation	Clean and preprocessed dataset; quality report.	Data Lead
2	Initial Training	First trained model; basic metrics.	Modeling Lead
3	Validation and Tuning	Cross-validation, hyperparameter tuning, and performance report.	Modeling Lead
4	Dashboards and Explainability	Result visualizations and explanation of predictions.	Explainability & Reporting Lead
5	Final Documentation	Complete report, risk summary, and consolidated timeline.	Documentation & Project Management Lead

Table 3: Five-week plan: milestones, deliverables, and owners.

4.3 Methodology or management tool used

Kanban methodology will be applied using platforms such as Trello or Miro, due to their ease of organizing tasks and visualizing the workflow.

Main advantages:

- Clear visibility into assigned tasks and their current status.
- Flexibility to adapt to changes.
- Rapid identification of blockers or delays.
- Transparent progress tracking for the entire team.

4.4 Schedule and workflow

The schedule is visualized using a Gantt-style diagram, showing the main tasks and their estimated durations:

#	Activity / Milestone	Start date	End date	Week 1	Week 2	Week 3	Week 4	Week 5
1	Data preparation and cleaning	10/11/2025	14/11/2025					
2	Initial model training	17/11/2025	21/11/2025					
3	Validation and model tuning	24/11/2025	28/11/2025					
4	Prediction interpretability	01/12/2025	05/12/2025					
5	Documentation and closure	10/12/2025	14/12/2025					

Table 4: Five-week schedule: activities, dates, and weekly progress.

Alternatively, it can also be visualized in a weekly activities format:

Week	Milestone	Deliverable
1	Data preparation and cleaning	Clean, preprocessed dataset; quality report with exploratory analysis (outliers, missing values, normalization).
2	Initial model training	First trained model (Random Forest or simple neural network), basic metrics logged.
3	Validation and model tuning	Cross-validation, hyperparameter tuning; model comparison; performance tracking and model versioning.
4	Prediction interpretability	Feature-importance analysis using SHAP/LIME; concise report with plots and notes on biological consistency.
5	Documentation and closure	Consolidated final report: risk summary, modeling decisions, interpretability results, final timeline; up-to-date repository with notebooks and scripts.

Table 5: Five-week schedule: milestones and deliverables.

5 Incremental Improvements

In this phase of the project, improvements were implemented in the architecture and management of the Mechanisms of Action (MoA) prediction system to strengthen its robustness, reliability, and sustainability. Although no external feedback was received, the team carefully reviewed previous designs and internally adjusted key aspects of the system.

Architecture refinement. The modules were reorganized to ensure each component

operates independently, facilitating testing, maintenance, and scalability. Explanatory notes were added to the architecture diagram to indicate how each module contributes to the system’s quality and reliability.

Risk and quality management. More precise mitigation strategies were defined for potential risks such as data preparation errors, model overfitting, and biological validation. Basic monitoring mechanisms were incorporated into each module to detect possible issues during development and operation.

Project management. Team roles, milestones, and deliverables were adjusted to improve internal organization and ensure traceability of changes. Documentation and version control on GitHub were also strengthened, guaranteeing reproducibility and coordination of the team’s work.

These improvements reflect the internal evolution of the design and project management, consolidating a more robust approach that is better prepared for the next phases of development and implementation.

6 Final Reflection

This phase of the workshop was a key moment for consolidating the understanding and application of robust engineering principles in system development. One of the most significant challenges was visualizing and structuring the architecture of the Mechanisms of Action (MoA) prediction system, given that earlier phases lacked a formal diagram. Creating this representation made it possible to identify interdependencies among modules, clarify each component’s responsibilities, and establish internal mechanisms to improve the system’s reliability, scalability, and maintainability. This process reaffirmed the importance of modularity and structured documentation as fundamental tools for efficiently managing complex projects.

Likewise, the risk analysis phase highlighted the need to anticipate potential points of failure, including errors in data preparation, model overfitting, and limitations in biological interpretability. Defining mitigation and monitoring strategies not only strengthened the system’s resilience but also prompted the team to reflect on how proactive planning and the adoption of quality standards such as ISO 9000, CMMI, and Six Sigma principles directly impact the effectiveness of outcomes.

Finally, this experience yielded valuable lessons on project management in complex development environments—from defining roles and milestones to organizing documentation and version control. Consolidating these practices not only improves the system’s reliability and sustainability, but also strengthens the team’s competencies in making informed decisions, anticipating risks, and iterating on existing designs.

7 Conclusions

- In this phase of the workshop, a modular, well-documented architecture was established for the Mechanisms of Action (MoA) prediction system, enabling each module to be executed, tested, and updated independently. Risk-monitoring and quality-control mechanisms were implemented to strengthen the system’s reliability and robustness against data errors, model overfitting, and limitations in biological interpretability.
- Clear roles, milestones, and deliverables were defined for the team, along with centralized version control in GitHub. This facilitates coordination, change tracking, and collaboration among team members, ensuring a more organized, reproducible, and sustainable development process.
- The improvements implemented (modular architecture, risk mitigation, and structured project management) ensure that the system is viable, reliable, and sustainable over the long term, preparing the team for subsequent implementation phases and continuous optimization. This approach ensures the project can adapt to new requirements and remain operational without compromising result quality.

8 References

- International Organization for Standardization. (2015). *Quality management systems — Fundamentals and vocabulary (ISO 9000:2015)*.
- CMMI Institute. (2018). *CMMI for Development, Version 2.0*.
- Pande, P. S., Neuman, R. P., & Cavanagh, R. R. (2000). *The Six Sigma Way*. McGraw-Hill.
- Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.
- Kaggle. (2025). *[Specific dataset name]*. <https://www.kaggle.com>